
D4.1 - NGSOTI architecture document

NGSOTI Project: 101127921
DIGITAL-ECCC-2022-CYBER-03

Team CIRCL/NGSOTI



Co-funded by
the European Union



ECCC 
EUROPEAN CYBERSECURITY
COMPETENCE CENTRE

2025-09-24

Contents

1	NGSOTI architecture	2
1.1	Disclaimer	2
1.2	Distribution and License	2
1.3	Deliverable Definition	2
2	NGSOTI – Architecture Overview	2
2.1	Scope	3
2.2	Purpose of the Architecture	4
2.3	Layered Design	4
2.4	Component Details	5
2.4.1	SkillAegis & CEXF	5
2.4.2	Kunai	5
2.4.3	Zeek	5
2.4.4	Suricata	6
2.4.5	Sysmon	6
2.4.6	Tenzir	6
2.4.7	Poppy	6
2.4.8	MISP	7
2.4.9	Vulnerability-Lookup	7
2.4.10	D4 Project	7
2.4.11	FlowIntel	7
2.4.12	AIL Framework	8
2.4.13	Lookyloo	8
2.4.14	Pandora	8
2.4.15	Cerebrate	8
2.5	Measuring usability and the user experience and impact	9
2.6	Example End-to-End Data Flow	9
2.6.1	Sample exercise workflow	9
2.6.2	Additional Exercise Workflow: ISAC Intelligence to SOC Action	10
2.7	Conclusion	10

1 NGSOTI architecture

1.1 Disclaimer

Co-funded by the European Union. Views and opinions expressed are however those of the author(s) only and do not necessarily reflect those of the European Union or the European Cybersecurity Competence Centre. Neither the European Union nor the granting authority can be held responsible for them.

Words displayed in red indicate hyperlinks. These links can be clicked or viewed by hovering over the text.

1.2 Distribution and License

The document is distributed under Creative Common Attribution 4.0 International [CC-BY](#).

The document is distributed as TLP:CLEAR.

1.3 Deliverable Definition

The identifier of the deliverable is D4.1 and it adheres to the definition outlined in the grant agreement written in bold. **NGSOTI architecture document**. The description is: **Blog post of the NGSOTI architecture and exploitation to collect data on [d4-project.org](#)**.

The blogpost in this deliverable was published the 19th September 2025 on [d4-project.org](#)

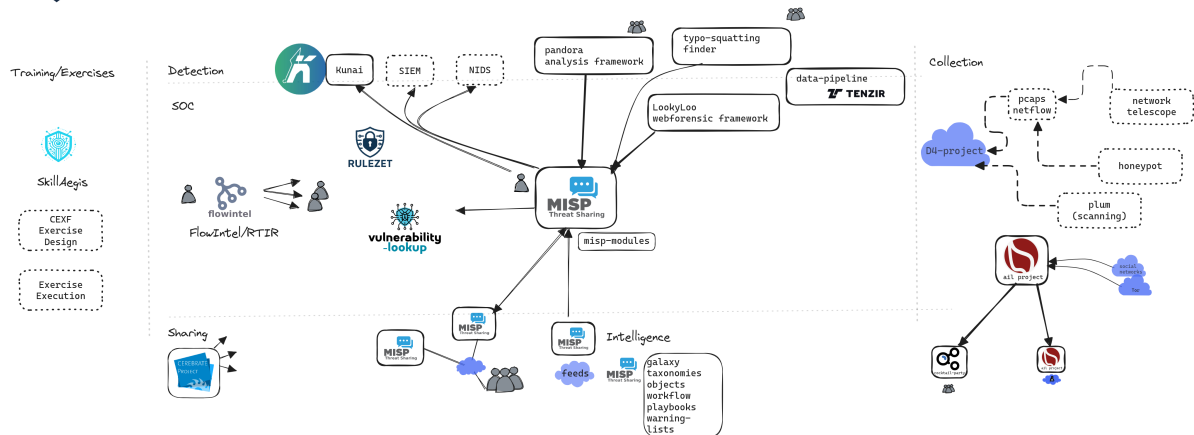
2 NGSOTI – Architecture Overview

The Next Generation SOC Training Infrastructure ([NGSOTI](#)) [project](#) is an open-source initiative to build realistic, reproducible SOC environments for training and education. It integrates mature open-source components with **ReST APIs and OpenAPI specifications** to ensure interoperability, scalability, and extensibility.

This document aims to present a comprehensive overview of the NGSOTI architecture while also reflecting on the insights, feedback, and experience gained during the initial phase of the project. The NGSOTI Architecture overview is shown the figure below:



NGSOTI - Architecture Overview



2.1 Scope

The NGSOTI project is designed to replicate the day-to-day environment of a modern Security Operations Center (SOC) and provide trainees with the essential skills required to operate effectively in such settings. To achieve this, the training infrastructure must cover the core requirements of SOC analysts and incident responders:

- **Log Management and Analysis.** SOC analysts must be able to collect, normalise, store, and query large volumes of heterogeneous logs (endpoint, network, application, cloud). Training scenarios need to expose students to real log pipelines, from raw ingestion to search and correlation.
- **False-Positive Handling and Triage.** Analysts must learn to distinguish between benign anomalies and true security incidents. Training must include workflows for alert triage, suppression of recurring false positives, and case escalation.
- **Detection Engineering.** SOC teams are not only consumers of alerts but also creators of new detection rules. Training requires environments where students can experiment with creating, testing, and deploying new signatures, queries, or behavioural detections across different telemetry sources.
- **Information Sharing and Collaboration.** Modern SOC's rarely work in isolation. They rely on information exchange with other teams and communities. Exercises must therefore include mechanisms for sharing Indicators of Compromise (IOCs), Tactics, Techniques, and Procedures (TTPs), and incident reports in structured formats.
- **Threat Intelligence Integration.** To contextualise alerts and drive better decision-making, analysts need access to threat intelligence feeds and vulnerability information. Training environments must integrate open-source threat intelligence platforms (e.g., MISP) and vulnerability

knowledge bases (e.g., Vulnerability-Lookup) to provide authentic enrichment and prioritisation workflows.

By embedding these requirements into the training scope, NGSOTI ensures that students are not only exposed to realistic datasets and tools, but also to the **operational processes** and **analytical workflows** that define real-world SOC operations. This explains the architectural choices described in the following sections.

2.2 Purpose of the Architecture

NGSOTI addresses key challenges in SOC training:

- **Infrastructure Access:** Many training centers/universities lack resources to simulate SOC environments.
- **Integration with Modern Tooling** SOC tools evolve quickly; training must adapt.
- **Reusable Exercises:** Scenarios must be standardised (via CEXF) for replay and sharing.
- **Accessible Datasets:** Trainees should work with realistic telemetry and dataset from real-world sources.

2.3 Layered Design

The table below gives an overview of layers and their associated components

Layer	Components
Trainee & Instructor	SkillAegis
Telemetry	Kunai, Zeek, Suricata, Sysmon
Data Pipeline	Tenzir, Poppy
Threat & Vulnerability Intelligence	MISP, Vulnerability-Lookup
Collection	D4 Project, Honeypots, Network Telescope
Storage	NFS, S3 / MinIO
Analyst Tooling	FlowIntel, RTIR, Wazuh dashboards
Dark Web & OSINT	AIL
Forensics & Enrichment	Lookyloo, Typo-squatting finder, Pandora
Sharing	Cerebrate

2.4 Component Details

It is important to note that the components described in this section represent the **current set of integrated tools**, but the architecture is not limited to these specific projects. The NGSOTI stack is intentionally designed to be **modular and extensible**, making it possible to replace, extend, or complement components as new tools and technologies emerge. This means that future iterations of the architecture document can evolve to incorporate additional open-source projects, new detection capabilities, or alternative analyst platforms, while still adhering to the same design principles of interoperability, open standards, and reproducibility. The components described in this section represent the current set of integrated tools, but the architecture is not limited to these specific projects. Since the original proposal, the architecture has matured to reflect both the core technical stack and additional elements that strengthen operational use and training.

2.4.1 SkillAegis & CEXF

- **Purpose:** Web portal for instructors and trainees. Defines and executes exercises in Common Exercise Format (CEXF). Provides live scoreboard and feedback.
- **Inputs:** Exercise definitions (JSON), datasets, injects.
- **Outputs:** Exercise deployment instructions, scoreboard updates.
- **APIs:** ReST API (OpenAPI).
 - ReST API (OpenAPI)
- **Integration:** Pulls case status from FlowIntel, IOC/CVE artefacts from MISP, and telemetry triggers from Tenzir.

2.4.2 Kunai

- **Purpose:** Lightweight Linux sensor using eBPF for syscall and network visibility.
- **Inputs:** Kernel-level telemetry.
- **Outputs:** JSON structured logs (syscalls, processes, sockets, files).
- **APIs:** gRPC & ReST endpoints to push events into pipelines. Online sandbox is available.
- **Integration:** Feeds events directly into Tenzir for enrichment and storage or MISP event.

2.4.3 Zeek

- **Purpose:** Network traffic analysis framework for metadata-rich logs.
- **Inputs:** PCAPs, live network traffic.
- **Outputs:** Connection logs, DNS, HTTP, SSL/TLS metadata in structured log format.
- **APIs:** Exposes logs via file or ReST exporter plugins.

- **Integration:** Tenzir ingests Zeek logs for correlation and enrichment. Integration with MISP.

2.4.4 Suricata

- **Purpose:** IDS/IPS engine with signature-based traffic inspection.
- **Inputs:** PCAPs, live network traffic.
- **Outputs:** Alerts, flow records, protocol metadata (EVE JSON).
- **APIs:** Native EVE JSON output; ReST management API.
- **Integration:** Suricata alerts streamed into Tenzir → Wazuh dashboards. Integration with MISP and also SkillAegis.

2.4.5 Sysmon

- **Purpose:** Endpoint monitoring on Windows. Provides detailed process, file, and registry telemetry.
- **Inputs:** Windows system events.
- **Outputs:** XML/EVTX logs → converted into JSON.
- **Integration:** Shipped to Tenzir pipeline for enrichment with MISP/ Vulnerability-Lookup.

2.4.6 Tenzir

- **Purpose:** Data pipeline engine for high-throughput collection, transformation, enrichment, and routing.
- **Inputs:** Zeek, Suricata, Kunai, Sysmon logs.
- **Outputs:** Normalised events, enriched detections, SIEM dashboards, storage.
- **APIs:** ReST API (OpenAPI) for managing pipelines (/pipeline/start, /pipeline/stop, /pipeline/status). Query endpoints for structured data (/query).
- **Integration:** Pulls IOCs from MISP. Queries CVE data from Vulnerability-Lookup. Uses Poppy for fast IOC set checks.

2.4.7 Poppy

- **Purpose:** Bloom filter engine for efficient set membership testing.
- **Inputs:** IOC sets (IP, hash, domain).
- **Outputs:** Boolean match/no-match on streaming telemetry.
- **APIs:** [Library](#).
- **Integration:** Embedded in Tenzir pipelines for inline detection.

2.4.8 MISP

- **Purpose:** Threat Intelligence sharing platform.
- **Features:** IOC sharing (domains, IPs, hashes). Galaxies, taxonomies, objects, workflows, playbooks, warning-lists. Automated enrichment via misp-modules.
- **Inputs:** IOCs from AIL, Lookyloo, Pandora, Cerebrate, external feeds.
- **Outputs:** Events, correlations, sightings, enriched attributes.
- **APIs:**
 - ReST API ([OpenAPI](#)).
 - Feed sync APIs for federation.
- **Integration:** Queried by Tenzir for enrichment. Injects IOCs/CVEs into SkillAegis exercises. Synced across training centers via Cerebrate.

2.4.9 Vulnerability-Lookup

- **Purpose:** Aggregates CVE data, vendor advisories, EPSS, and Vuln4Cast predictions.
- **Inputs:** NVD CVE feeds, vendor advisories, predictive datasets.
- **Outputs:** Vulnerability metadata, risk scores, exploit predictions.
- **APIs:** ReST API ([OpenAPI](#)).
- **Integration:** Queried by Tenzir for contextual enrichment. Linked into FlowIntel cases. Provides CVE injection for SkillAegis.

2.4.10 D4 Project

- **Purpose:** Provides authentic pcap and netflow datasets.
- **Inputs:** Network sensors, honeypots, telescope feeds.
- **Outputs:** Historical and live traffic data.
- **APIs:** Dataset download API + ReST endpoints for metadata.
- **Integration:** Replayed into Zeek/Suricata/Kunai for training scenarios. Raw materials available for student use and SOC processing.

2.4.11 FlowIntel

- **Purpose:** Case management platform for SOC investigations.
- **Inputs:** Alerts from Tenzir, Wazuh dashboards, MISP correlations.
- **Outputs:** Cases, workflows, analyst notes.
- **APIs:** ReST API ([OpenAPI](#)) for case lifecycle.

- **Integration:** Linked to SkillAegis to update scoreboard. Receives MISP IOCs for case enrichment.

2.4.12 AIL Framework

- **Purpose:** Dark web & OSINT collection framework.
- **Inputs:** Social networks, Tor hidden services, paste sites, leaks.
- **Outputs:** Extracted IOCs, credentials, malware samples.
- **APIs:** [ReST API](#)
- **Integration:** Feeds directly into MISP for correlation and exercise design. Alerts can be fed into FlowIntel.

2.4.13 Lookyloo

- **Purpose:** Web forensic capture and analysis tool.
- **Inputs:** URLs/domains.
- **Outputs:** DOM tree, tracker info, TLS certificates, screenshots.
- **APIs:** [ReST API](#), [OpenAPI documentation](#).
- **Integration:** Enrichment service for MISP events and SkillAegis injects.

2.4.14 Pandora

- **Purpose:** File/malware static analysis and enrichment framework.
- **Inputs:** Samples (executables, scripts, documents).
- **Outputs:** Behavioural analysis reports, indicators.
- **APIs:** [ReST API](#), [pypandora API reference](#).
- **Integration:** Pushes enriched artefacts into MISP.

2.4.15 Cerebrate

- **Purpose:** Federation and community management layer.
- **Inputs:** MISP instances, training centers, partner metadata.
- **Outputs:** Synchronised datasets, directories of trusted orgs.
- **APIs:** [ReST API \(OpenAPI\)](#).
- **Integration:** Distributes MISP data, exercises, and configurations across institutions.

2.5 Measuring usability and the user experience and impact

Following objective WP3.O3 (‘Collect feedback from existing trainings to improve future ones’), training activities will be continuously improved throughout the project whenever new, relevant methodologies emerge and can be practically applied in the context of NGSOTI. **Usability is measured** by whether users can complete their tasks effectively and without errors, using the SkillAegis tool. SkillAegis determines whether tasks were executed without errors and how many errors students made. It also collects metadata on efficiency, such as how quickly tasks are completed. **User experience** is an important source of improvement for the NGSOTI tools. Feedback on user experience is translated into GitHub issues that document proposed enhancements. User experience is assessed by trainers and by users who open issues in the respective GitHub repositories. Within NGSOTI, **impact** can be assessed using two complementary indicators:

- (1) the number of GitHub stars across project repositories, including both software tools and training documentation; A GitHub star can be understood as an endorsement and bookmarking mechanism for a software repository. When a user stars a repository, it typically indicates that they consider it relevant or useful, that they wish to retain it for future reference, and/or that they intend to express support for the project, comparable to a simple positive vote.
- (2) the uptake of NGSOTI components through their integration into training materials delivered by third parties.

2.6 Example End-to-End Data Flow

The architecture has been designed and validated using real scenario flows, as outlined in the exercise workflow section. By mapping each component of the stack to the practical steps of a training exercise, from scenario creation in SkillAegis, through real-data detection and enrichment in Tenzir and MISP, to case handling in FlowIntel, we ensured that the system is not only conceptually sound but also operationally effective. This validation process demonstrated that the architecture can support realistic SOC analyst activities, such as log analysis, detection engineering, triage, and intelligence-driven decision making, thereby confirming its suitability for real-world training environments including the infrastructure from private training centers or University such as the University of Luxembourg or University of Lorraine.

2.6.1 Sample exercise workflow

1. Instructor creates an exercise in SkillAegis (CEXF).
2. Trainee actions generate detection from Kunai, Zeek, Suricata, Sysmon.
3. Tenzir collects → normalises → enriches with MISP and Vulnerability-Lookup → filters via Poppy.

4. Alerts appear in Wazuh dashboards, all data archived in object storage.
5. Confirmed alerts create cases in FlowIntel; scoreboard updated in SkillAegis.
6. Continuous enrichment: AIL, Lookyloo, Pandora feed into MISP.
7. Cerebrate synchronises intelligence and exercises across training centers.

2.6.2 Additional Exercise Workflow: ISAC Intelligence to SOC Action

1. **Intelligence Ingestion.** An ISAC shares a new MISP event containing indicators related to an active campaign. The event is automatically synchronised into the training environment's MISP instance through federation.
2. **Exercise Setup.** The instructor assigns the scenario in SkillAegis, which provides the trainee with access to the relevant MISP event, including IOCs (domains, hashes, IP addresses) and contextual information (threat actor, campaign, TTPs).
3. **Detection and Correlation.** The trainee must configure detection rules in Tenzir, Suricata, or Zeek to monitor lab telemetry for signs of the shared indicators. This step requires hands-on detection engineering to ensure that the SOC tooling reacts to the threat intelligence feed.
4. **Analysis and Case Handling.** If matches are found in the telemetry, alerts are generated and forwarded to FlowIntel. The trainee is responsible for triaging the case, reviewing logs, and validating whether the activity represents malicious behaviour or false positives.
5. **Outcome Resharing.** Once the analysis is complete, the trainee must update the original MISP event with additional findings (new IOCs, sightings, correlations) and reshare the enriched event back into the ISAC community. This tests the information sharing and collaboration process in SOC operations.
6. **Feedback and Scoring.** SkillAegis retrieves case outcomes and event updates to calculate scores based on the quality of detections, accuracy of analysis, and completeness of the information shared back to the community.

2.7 Conclusion

The NGSOTI architecture is not intended as a rigid, one-size-fits-all solution. Instead, it is designed as a set of building blocks that can be assembled, adapted, or selectively deployed depending on the needs of a training center, university, or SOC exercise. Each component, from telemetry sensors to case management platforms, exposes documented ReST APIs and adheres to open standards, ensuring interoperability and reusability across diverse environments, including challenging training setups with rigid constraints or limited access to network services.

This modularity allows training organizers to cherry-pick the tools that best match their objectives. For example, one exercise may focus primarily on network traffic analysis, relying heavily on Zeek and Suricata, while another may emphasize case management and collaboration, making greater use of

FlowIntel and MISP. By decoupling the layers, the architecture supports both minimal deployments for lightweight scenarios and full-stack deployments for end-to-end SOC simulations.

Another advantage of this approach is progressive adoption. Institutions that may initially lack the infrastructure or expertise to run a full SOC stack can start small, integrating just a subset of components. Over time, they can expand their setup by adding new datasets, enrichment modules, or analyst tooling, without having to rebuild the entire environment. This incremental path lowers the entry barrier and fosters long-term sustainability.

Equally important, the use of open-source projects ensures that the tools are transparent, extensible, and supported by active communities. Trainers and students alike benefit from engaging with widely adopted platforms such as MISP, Tenzir, and Suricata, gaining skills directly transferable to professional SOC environments. At the same time, the open ecosystem allows contributors to extend functionalities and share improvements, creating a virtuous feedback loop.

In summary, the NGSOTI architecture provides a flexible foundation for SOC training. It offers a common framework that can be tailored to diverse requirements, encourages reuse of scenarios and data, and lowers the barriers to creating authentic training environments. Whether deployed in its entirety or in parts, NGSOTI empowers institutions to deliver realistic, modern, and effective SOC training experiences.